

Chapter 1: The Anatomy of Python

Topic 1: The Lexer & Tokenization

When you write Python code, the computer doesn't see "code"; it sees one giant, continuous string of text characters. Before Python can execute your script, it must pass the text through a specialized component called the **Lexer** (short for lexical analyzer).

***Tokenization** is the process of breaking that long string of text down into the smallest possible meaningful units, which we call **tokens**. Think of it like reading a sentence: your brain automatically breaks down a line of text into individual words and punctuation marks so you can understand the grammar.*

- **The Analogy:** If your code is a brick wall, tokens are the individual bricks. A pile of loose clay means nothing until it is molded into separate bricks.
- **Why it matters:** If you type something the lexer doesn't recognize (like a random symbol \$ where it shouldn't be), Python throws a `SyntaxError` immediately before it even tries to run the program.

2. Live Code

Let's see exactly how Python views a simple line of code.

The Code:

```
x = 10 + 5
```

The Lexer's Break Down:

Python turns that single line into five distinct tokens:

1. `x` (Name / Identifier)
2. `=` (Assignment Operator)
3. `10` (Number Literal)
4. `+` (Arithmetic Operator)
5. `5` (Number Literal)

3. Watch Out! (Common Gotchas)

- **Spaces Matter, But Mostly for Readability:** `x=10+5` and `x = 10 + 5` generate the exact same tokens. The lexer usually throws away normal whitespace *between* tokens.
- **The Indentation Exception:** Python is unique because spaces at the *very beginning* of a

line actually do create special tokens (INDENT and DEDENT). Messing up your starting spaces will confuse the lexer completely.

4. Topic 1 Practice Questions

- 1. What is the main job of the Lexer in Python?
- 2. What do we call the smallest meaningful units of code that the lexer creates?
- 3. Look at this line: total = price * 2. List out every single token the lexer will generate.
- 4. If you write code with an illegal character that Python doesn't recognize, what specific type of error will you get?
- 5. Does the lexer treat a+b differently than a + b? Why or why not?

Topic 2: The 5 Pillars of Syntax

Every single token the lexer extracts fits cleanly into one of five functional categories. We call these the **5 Pillars of Syntax** . Knowing these categories helps you read code like a map.

Category	What it is	Everyday Example
1. Identifers	Names you invent to label things (variables, functons, classes).	age, calculate_total, user_name
2. Keywords	Reserved words that already mean something specifc to Python. You cannot change them.	if, else, for, while, def, True
3. Literals	Raw data values typed directly into the code.	42 (integer), "Hello" (string), 3.14 (foat)
4. Operators	Symbols that perform actons or calculatons on data.	+, -, *, /, ==, >
5. Delimiters	Punctuaton marks used to group, separate, or organize code boundaries.	(), [], { }, , ;, =

- Open your terminal or VS Code and look at how these pillars interact:

A simple block using all 5 pillars

```
score = 100

if score > 90:

    print("Passed")
```

Deconstruction:

- **Identifiers:** score, print
- **Keywords:** if
- **Literals:** 100, 90, "Passed"
- **Operators:** >, =
- **Delimiters:** :, (,)

Tokenization

3. Watch Out! (Common Gotchas)

- **Keyword Hijacking:** You cannot name a variable after a keyword. Typing `if = 5` or `True = "Yes"` will crash your program instantly.
- **Case Sensitivity:** Python is incredibly strict about capitalization. `if` is a keyword; `If` or `IF` are treated as custom identifiers (variables), which can lead to confusing bugs.

4. Topic 2 Practice Questions

1. Name the 5 Pillars of Syntax.
2. Identify the category for each token in this line: `status = True`.
3. Why will the line of code `for = 10` cause an error?
4. Is "25" considered a literal or an identifier? Explain why.
5. What category do square brackets `[]` and colons `:` belong to?

Topic 3: Variable Name Tags

*Most programming languages teach you that a variable is a "box" that holds data. **Forget the box metaphor.** In Python, it is fundamentally incorrect and will confuse you later.*

In Python, variables are **Name Tags** (or pointers).

- When you write `x = 10`, Python doesn't put the number 10 inside a box named x.
- Instead, Python creates the object 10 out in a space called memory, and then ties a string labeled "x" directly to it.

The Multi-Tag Reality: Because variables are just name tags, you can stick multiple name tags onto the exact same object.

3. Watch Out! (Common Gotchas)

- **The Illusion of Copying:** Writing `b = a` does *not* make a copy of the value. It just duplicates the pointer. If the object itself changes later (which happens with complex structures like lists), both variables will reflect that change!

4. Topic 3 Practice Questions

1. Why is the “box” metaphor inaccurate for Python variables?
2. What is a more accurate real-world metaphor for a Python variable?
3. If you execute `x = 7` and then `y = x`, how many copies of the number 7 exist in memory?
4. If you run the code `score = 50` followed by `score = 60`, what happens to the number 50 in memory?
5. Draw or describe the memory layout after these three lines run:

```
m = "Python"
```

```
n = m
```

```
p = m
```

Topic 4: Memory Identification (`id()`)

Every single thing created in Python gets assigned a unique memory address—think of it like a permanent GPS coordinate or a home address inside your computer’s RAM.

Python gives us a built-in tool called `id()` to look up this address.

- `id(variable)` returns a long number representing the exact location of the object.
- **Identity vs. Value Equality:** This brings up a critical rule in Python programming:
 - `==` checks if two things have the **same value** (Are they twins?).
 - `is` checks if two things have the **same identity address** (Are they literally the exact same physical person?).

(Terminal Verification)

Let’s use the terminal to verify these memory addresses directly.

```
list1 = [1, 2, 3]
list2 = [1, 2, 3]

# Check values

print(list1 == list2) # Output: True (Values match)

# Check identities

print(id(list1)) # Output: 140234857211008 (example address)

print(id(list2)) # Output: 140234857211456 (different address!)

print(list1 is list2) # Output: False
```

3. Watch Out! (Common Gotchas)

- **Assuming == means is:** Just because two variables yield the exact same output when printed doesn't mean they point to the same memory slot. Always use == for comparing data values, and reserve is purely for checking if two things are the exact same object in memory.

4. Topic 4 Practice Questions

1. What does the built-in id() function tell us about a variable?
2. Explain the difference between the == operator and the is operator.
3. If id(a) == id(b) is true, what will a is b return?
4. Suppose you create two completely separate strings with the exact same text contents. Will they always share the same id()? (Think carefully about value vs identity).
5. Look at this code:

```
x = [10, 20]
y = x
```

Will x is y evaluate to True or False? Why?

Topic 5: The Object System

*In many programming languages, simple numbers like 5 or letters like 'A' are raw, basic bits of data. Python handles things differently: **Everything in Python is an Object.***

Even a basic integer like 5 is a fully packaged object containing metadata wrapped around the raw value. Every single object in Python carries at least three elements:

1. **An Identity Address:** Its location in memory (retrieved via `id()`).
2. **A Type:** What class or category of data it belongs to (retrieved via `type()`).
3. **A Value:** The actual data it represents.

Because integers and strings are full objects, they come pre-packaged with built-in internal tools (called methods) that you can run directly on them.

Let's inspect an ordinary number using the terminal to see its underlying object nature:

```
x = 42

print(type(x)) # Output: <class 'int'>

# Let's see how many bits it takes to represent this number

print(x.bit_length()) # Output: 6 (42 in binary is 101010)
```

3. Watch Out! (Common Gotchas)

- **Overhead Costs:** Because everything is an object containing hidden tracking data, Python uses slightly more computer memory than lower-level languages (like C or C+). This trade-off is what makes Python so flexible and user-friendly.

4. Topic 5 Practice Questions

1. What three pieces of metadata does every single object in Python contain?
2. What function do you use to check the data type of an object?
3. True or False: A simple integer like 7 has built-in methods you can interact with.
4. What class type does Python assign to a decimal number like 9.99?
5. Why does Python use a bit more memory to store basic data compared to lower-level languages like C?

Topic 6: Small Object Interning

*creating objects takes up memory and processing power, Python applies an optimization trick behind the scenes called **Small Object Interning**.*

When Python starts up, it automatically creates and stores a pool of specific objects in memory:

- **Integers ranging from -5 to 256**

Whenever you use a number within this narrow range anywhere in your script, Python won't waste time creating a new object. Instead, it seamlessly hands you a name tag pointing directly to the shared number already waiting in the pool.

Terminal Verification

Let's test this in the terminal. Watch how the IDs match for small numbers but diverge for larger ones.

Test within the interned range (-5 to 256)

```
a = 100

b = 100

print(a is b) # Output: True (They point to the exact same pre-built object!)

# Test outside the interned range

x = 300

y = 300

print(x is y) # Output: False (Python created two separate 300 objects in memory)
```

tokens

3. Watch Out! (Common Gotchas)

- **Terminal vs. Script Behavior:** If you run the code above as a completed script file in VS Code rather than typing it line-by-line in a terminal, Python's compiler will optimize the whole file at once and make `x is y` return `True`. For a reliable test of basic interning behavior, run these checks line-by-line inside a live interactive Python terminal.

4. Topic 6 Practice Questions

1. What is small object interning?
2. What is the exact range of integers that Python automatically pre-allocates in memory?
3. If you run `g = 5` and `h = 5` in the interactive terminal, will `g is h` be `True` or `False`?
4. If you run `m = 500` and `n = 500` line-by-line in the terminal, why does `m is n` evaluate to `False`?
5. What is the primary reason Python uses interning for small numbers?

Topic 7: Dynamic vs. Strong Typing

Python's data management system relies on two core design choices:

1. **Dynamic Typing:** You do not need to explicitly announce what type of data a variable holds ahead of time. The type is tied directly to the **object itself**, not the name tag. A tag can switch from pointing to a number to pointing to a string at any time.

2. **Strong Typing:** Python strictly respects data boundaries. It will never silently guess or force incompatible types to mix. If you try to combine a string and a number without explicit conversion, Python stops and raises a `TypeError`.

Let's look at both concepts in action:

Proof of Dynamic Typing

```
data = 10

print(type(data)) # Output: <class 'int'>

data = "Hello"    # The name tag easily switches to a string object

print(type(data)) # Output: <class 'str'>

# --- Proof of Strong Typing ---

price = 50

tax = "5"

# This will crash! Python refuses to guess if you want 55 or "505"

print(price + tax) # Output: TypeError: unsupported operand type(s) for +: 'int' and
```

tokens

3. Watch Out! (Common Gotchas)

- **Type Confusion:** Dynamic typing gives you incredible freedom, but it demands careful tracking. If you accidentally reuse a variable name tag intended for a list and overwrite it with a number, you won't get an immediate error until you try to use it like a list again later down the line.

4. Topic 7 Practice Questions

1. What does it mean that Python is a "dynamically typed" language?
2. What does it mean that Python is a "strongly typed" language?
3. If you write `x = "10" + 5`, what will happen when Python runs that line?

4. How would you fix the code `total = 100 + "20"` so it executes successfully to equal 120?
5. True or False: In Python, data types belong to the variable names themselves rather than the underlying objects.